



《计算机图形学基础》

习题课2

助教 陈拓

2025年3月22日



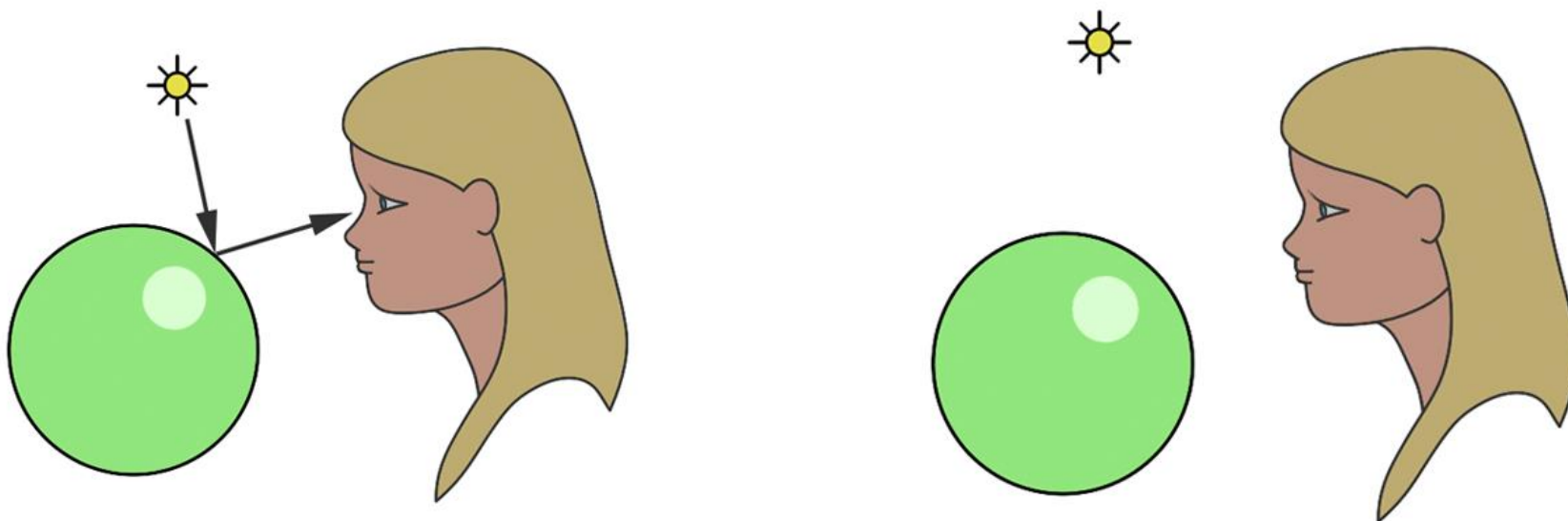
主要内容

- **PA1: 光线投射**
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分：蒙特卡洛积分
 - 终止：RR（Russian Roulette）
 - 对光源采样：NEE（Next Event Estimation）
 - 复杂采样方法：MIS（多重重要性采样）
- 大作业说明
 - 得分项简述
 - 评分细则



光线投射

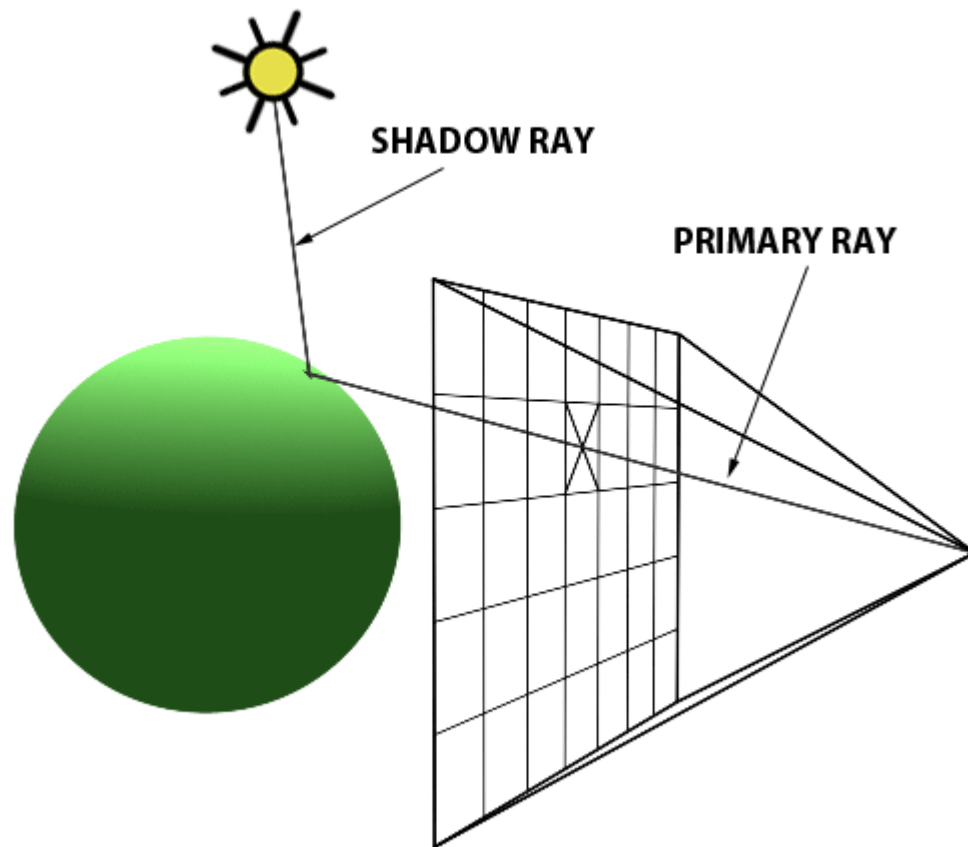
- 人为什么能看到物体？





光线投射

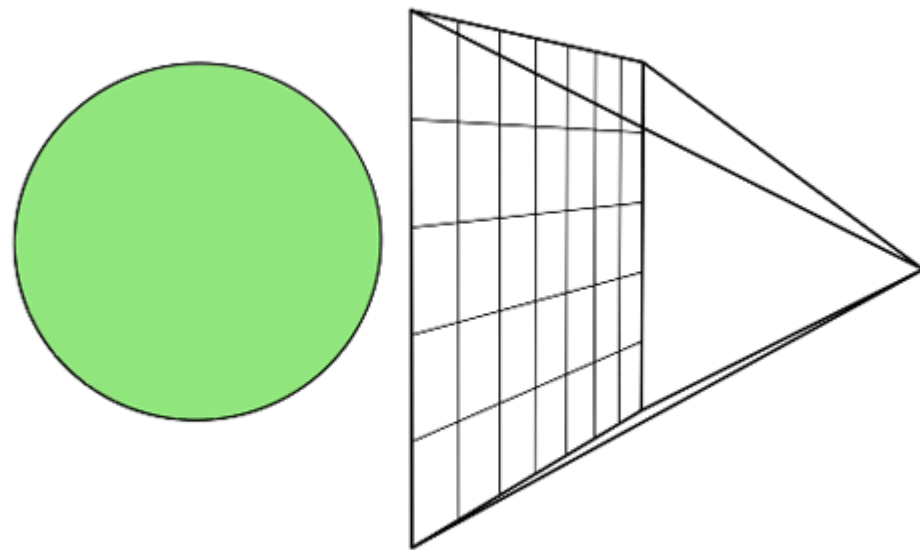
- 从视点出发逆向追踪光路





光线投射

- 循环所有像素：
 - 构建从相机出发，相机->像素方向的光线
 - 可以先构造相机空间光线方向，再变换到世界空间
 - 对场景物体求交点
 - 如果有交，通过Phong模型求出对应颜色
 - 没有交点的位置设为背景色





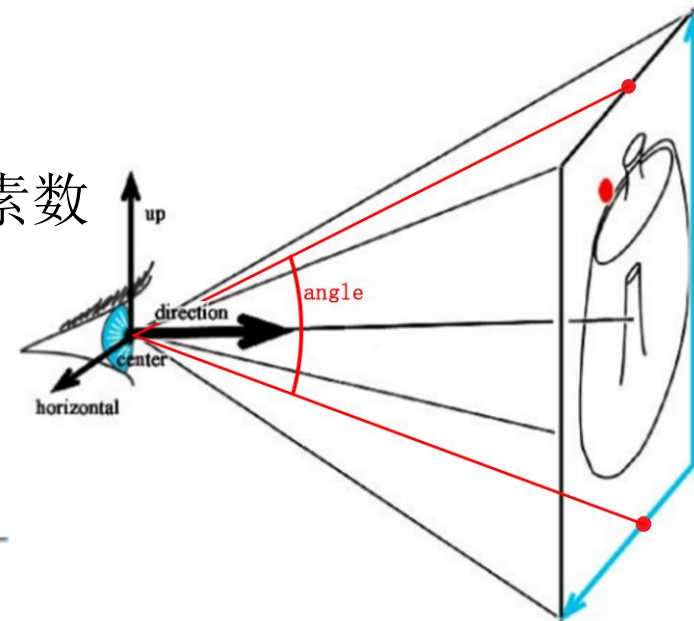
透视相机 - 构造光线

- 相机内参：
 - 画布（图像）大小 (w, h)
 - 光心位置 (c_x, c_y) ，PA中为 $(w/2, h/2)$
 - 视场角 $angle$
- 相机外参：
 - 视点位置 t
 - 视点朝向（指向光心） $direction$
 - 画布的水平轴 $horizontal$ 和垂直轴 up

- 给定图像坐标 (u, v) ，求视线方向 $\overrightarrow{d_{R_c}}$ ：
 - f_x, f_y 表示真实空间中画布上1单位距离对应图像中的像素数

$$f_x = f_y = \frac{h}{2 * \tan\left(\frac{angle}{2}\right)}$$

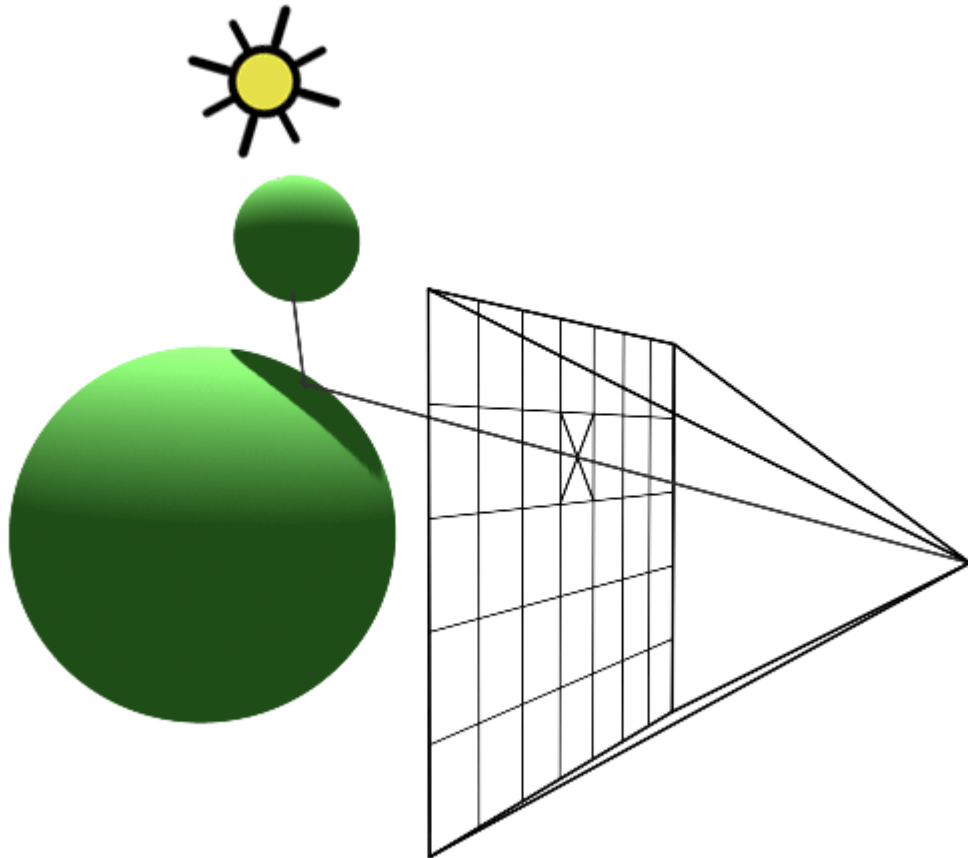
$$\overrightarrow{d_{R_c}} = \text{normalized}\left(\frac{u - c_x}{f_x}, \frac{c_y - v}{f_y}, 1\right)^T$$





光线投射 – 阴影

- 只计算光源对交点的直接贡献
- 作业中不考虑光源被遮挡的问题

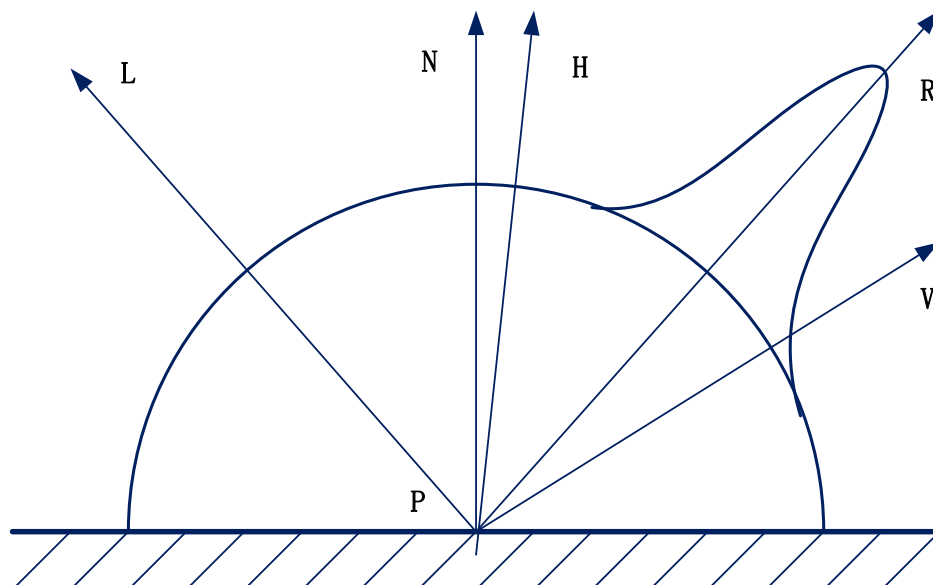




光线投射 - 着色

- Phong模型
- 视角方向接收的发光强度为环境光分量、镜面反射光分量和漫反射光分量之和
- 这次作业不要求实现环境光

$$I = I_i K_a + I_i K_s * (R \cdot V)^n + I_i K_d * (L \cdot N)$$





主要内容

- **PA1: 光线投射**
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分：蒙特卡洛积分
 - 终止：RR（Russian Roulette）
 - 对光源采样：NEE（Next Event Estimation）
 - 复杂采样方法：MIS（多重重要性采样）
- 大作业说明
 - 得分项简述
 - 评分细则



思考：如何拆解框架？

我们想做的事情包括？

- 1、构建场景：如何读取txt文件定义的场景？
- 2、构造光线：应该如何定义一根光线？
- 3、光线求交：求交的对象是什么？交点应该保存什么信息？
- 4、求交结果上色并保存到图片中：如何进行上色？上色的图片应该如何保存？



思考：如何拆解框架？

1. 构建场景（为此，需要从文件中读取场景信息的SceneParser类）
2. 构造光线（为此，需要有Camera相机类、Ray光线类）
3. 光线求交（为此，需要有诸多物体类、Hit交点类、Material材质类）
 - 物体类：基类Object3D；基本几何体（plane、triangle、sphere）；复杂几何体（group、mesh、transform）
4. 求交结果上色并保存到图片中（为此，需要有Light光源类和Image图像类）
 - 光源类：方向光源类DirectionalLight；点光源类PointLight
5. 此外，还需要一些向量通用计算库（vecmath）



代码讲解 – 场景

- SceneParser类，读取输入的场景文件和模型
- 场景文件（scene*.txt）和模型文件 (*.obj)

```
Materials {  
    numMaterials 1  
    PhongMaterial {  
        diffuseColor 0.79 0.66 0.44  
        specularColor 1 1 1  
        shininess 20  
  
    }  
}  
  
Group {  
    numObjects 1  
  
    MaterialIndex 0  
    TriangleMesh {  
        obj_file mesh/bunny_200.obj  
    }  
}
```

```
v -1 -1 -1  
v 1 -1 -1  
v -1 1 -1  
v 1 1 -1  
v -1 -1 1  
v 1 -1 1  
v -1 1 1  
v 1 1 1  
f 1 3 4  
f 1 4 2  
f 5 6 8  
f 5 8 7  
f 1 2 6
```



代码讲解 – 物体

- Object3D类
- 所有物体类型的基类
- 需要在派生类中实现intersect求交函数

```
// Base class for all 3d entities.
class Object3D {
public:
    Object3D() : material(nullptr) {}

    virtual ~Object3D() = default;

    explicit Object3D(Material *material) {
        this->material = material;
    }

    // Intersect Ray with this object. If hit, store information in hit structure.
    virtual bool intersect(const Ray &r, Hit &h, float tmin) = 0;
protected:
    Material *material;
};
```



代码讲解

- 需要在派生类中实现intersect求交函数：
 - r表示视线射线，包含原点和方向
 - h用于记录并返回已遍历到的最近求交结果，返回(t, 材质, 法向)
 - tmin: 用于避免返回视线后方的交点，代码中tmin=0

```
bool intersect(const Ray &r, Hit &h, float tmin) override {  
    float t = ...;  
    if (t > tmin && t < h.getT()) {  
        h.set(t, material, norm);  
        return true;  
    } else  
        return false;  
}
```



代码讲解

- 对经过transform的物体求交
- 对一般的物体，我们希望求到视线 $\vec{o} + t\vec{d}$ 与物体的交点 $\vec{x}$ ，即 $\vec{x} = \vec{o} + t\vec{d}$
- 现在我们对物体上每个点做一个仿射变换，即 $\vec{x} \rightarrow A\vec{x} + \vec{b}$



代码讲解

- 对经过transform的物体求交
- 问题转变为求 $A\vec{x} + \vec{b} = \vec{o} + t\vec{d}$
- 实际上我们不需要真的对物体上每个点进行变换，因为上述式子可写成

$$\vec{x} = \left(A^{-1}\vec{o} - A^{-1}\vec{b} \right) + t(A^{-1}\vec{d})$$

- 换言之，我们只需要把视线起点改为 $A^{-1}\vec{o} - A^{-1}\vec{b}$ ，方向改为 $A^{-1}\vec{d}$ 对原物体求交



代码讲解

- 在Transform类中，我们正是对视线施加了逆变换来计算交点的。
- 注意，对视线施加了逆变换后，其方向向量**不一定是单位向量**，不能直接对其单位化，需要求这个非单位射线的对应t值：
- $\vec{x} = (A^{-1}\vec{o} - A^{-1}\vec{b}) + t(A^{-1}\vec{d})$
- 可以单位化求交后将t除以原向量长度。
- **单位化由函数调用者完成或者函数本身完成**都是可行的约定



代码讲解 – 材质

- Material类
- 定义了材质属性
- 需要根据Phong模型实现Shade函数

```
Vector3f Shade(const Ray &ray, const Hit &hit,  
               const Vector3f &dirToLight, const Vector3f &lightColor) {  
    Vector3f shaded = Vector3f::ZERO;  
    //  
    return shaded;  
}
```

protected:

```
Vector3f diffuseColor;  
Vector3f specularColor;  
float shininess;
```



代码讲解

- 根据Phong模型实现Shade函数：
 - ray表示视线射线
 - hit表示交点，含有法向等信息
 - dirToLight表示光线向量
 - lightColor表示光线颜色
 - 返回视线观察到的颜色值

```
Vector3f Shade(const Ray &ray, const Hit &hit,  
               const Vector3f &dirToLight, const Vector3f &lightColor) {  
    Vector3f shaded = Vector3f::ZERO;  
    //  
    return shaded;  
}
```



代码讲解 – 图像

- Image类
- 数组data存储颜色值

```
const Vector3f &GetPixel(int x, int y) const {  
    assert(x >= 0 && x < width);  
    assert(y >= 0 && y < height);  
    return data[y * width + x];  
}  
  
void SetAllPixels(const Vector3f &color) {  
    for (int i = 0; i < width * height; ++i) {  
        data[i] = color;  
    }  
}  
  
void SetPixel(int x, int y, const Vector3f &color) {  
    assert(x >= 0 && x < width);  
    assert(y >= 0 && y < height);  
    data[y * width + x] = color;  
}
```



代码讲解 – 计算库

- vecmath库
- 主要用到的是Vector3f类
- 重载了+-* /和[]访问
- 一些常用的函数



```
float length() const;  
float squaredLength() const;
```

```
void normalize();  
Vector3f normalized() const;
```

```
Vector2f homogenized() const;
```

```
void negate();
```

```
// ---- Utility ----
```

```
operator const float* () const; // automatic type conversion for OpenGL
```

```
operator float* (); // automatic type conversion for OpenGL
```

```
void print() const;
```

```
Vector3f& operator += ( const Vector3f& v );
```

```
Vector3f& operator -= ( const Vector3f& v );
```

```
Vector3f& operator *= ( float f );
```

```
static float dot( const Vector3f& v0, const Vector3f& v1 );
```

```
static Vector3f cross( const Vector3f& v0, const Vector3f& v1 );
```



主要内容

- PA1: 光线投射
 - 算法概述
 - 代码讲解
- 路径追踪
 - **PBR中如何计算渲染方程积分：蒙特卡洛积分**
 - 终止：RR（Russian Roulette）
 - 对光源采样：NEE（Next Event Estimation）
 - 复杂采样方法：MIS（多重重要性采样）
- 大作业说明
 - 得分项简述
 - 评分细则



Path Tracing

- 渲染的根本问题是什么？
- 渲染方程：

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega_+} L_i(p, \omega_i) f_r(p, \omega_i, \omega_r) (n \cdot \omega_i) d\omega_i$$

- 但是，如何解渲染方程呢？
 - 课程预告：数值分析
- 本质是估计一个积分
 - 方法：蒙特卡洛积分（Monte Carlo Integration）



蒙特卡洛积分

- 考虑定积分: $\int_a^b f(x)dx$
- 对于均匀随机变量 $X_i \sim p(x) = \frac{1}{b-a}$
估计为:
$$F_N = \frac{1}{N} \sum_{i=1}^N \frac{f(X_i)}{\frac{1}{b-a}} = \frac{b-a}{N} \sum_{i=1}^N f(X_i)$$
- 一般地, 对于非均匀随机变量 $X_i \sim p(x)$
估计为:
$$\langle F \rangle = \frac{1}{N} \sum_{i=1}^N \frac{f(X_i)}{p(X_i)}$$
- 无偏估计: 上式期望即等于所求定积分!



蒙特卡洛积分

- 蒙特卡洛估计量：
$$\int_a^b f(x)dx \approx \frac{1}{N} \sum_{i=1}^N \frac{f(X_i)}{p(X_i)}$$

- 渲染中，如何利用该方法求解渲染方程？

$$\int_{\Omega_+} L_i(p, w_i) f_r(p, w_i, w_o)(n \cdot w_i) dw_i \approx \frac{1}{N} \sum_{i=1}^N \frac{L_i(p, w_i) f_r(p, w_i, w_o)(n \cdot w_i)}{pdf(w_i)}$$

- 对入射光线方向 w_i 采样：也即采样光线
- w_i 方向接收到的radiance $L_i(p, w_i)$ ：也即下一层光线追踪的估计结果
- 修改 $f_r(p, w_i, w_o)$ ，如Cook-Torrance，得到glossy BRDF



主要内容

- PA1: 光线投射
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分: 蒙特卡洛积分
 - 终止: RR (Russian Roulette)
 - 对光源采样: NEE (Next Event Estimation)
 - 复杂采样方法: MIS (多重重要性采样)
- 大作业说明
 - 得分项简述
 - 评分细则



俄罗斯轮盘赌（Russian Roulette）

- 递归何时终止？
 - 递归深度？
 - 光强贡献？
- 我们希望得到一个**无偏**的结果！上述方法总会对更深层的光线欠考虑，因此总不能是无偏的
- 从数学上寻求解决手段.....
 - 每次迭代，以某个概率 q 终止光线
- 俄罗斯轮盘赌！



俄罗斯轮盘赌 (Russian Roulette)

- 我们用蒙特卡洛积分法，已经可以得到积分估计量 $\langle F \rangle$ 。
- 考虑这样一个量：

$$F' = \begin{cases} \frac{F - qc}{1 - q} & \xi > q \\ c & \text{otherwise.} \end{cases}$$

1 - q 的概率保持该采样。
但需要除以概率

以 q 的概率丢弃该采样

- c 通常取值为 0
- 这样，可以让期望仍为渲染方程积分；另一方面，期望递归深度变为至多 $1/(1-q)$



俄罗斯轮盘赌 (Russian Roulette)

shade(p, wo)

Manually specify a probability P_{RR}

Randomly select κ in a uniform dist. in $[0, 1]$

If ($\kappa > P_{RR}$) return 0.0;

Randomly choose ONE direction $w_i \sim \text{pdf}(w)$

Trace a ray $r(p, w_i)$

If ray r hit the light

Return $L_i * f_r * \text{cosine} / \text{pdf}(w_i) / P_{RR}$

Else If ray r hit an object at q

Return $\text{shade}(q, -w_i) * f_r * \text{cosine} / \text{pdf}(w_i) / P_{RR}$

图片来源: [GAMES101 Lec16](#)



主要内容

- PA1: 光线投射
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分: 蒙特卡洛积分
 - 终止: RR (Russian Roulette)
 - 对光源采样: NEE (Next Event Estimation)
 - 复杂采样方法: MIS (多重重要性采样)
- 大作业说明
 - 得分项简述
 - 评分细则



Next Event Estimation

- 路径追踪难以支持点光源——大作业中，请在代码框架里实现面光源
- 对于光源不够大的场景，路径追踪应用RR之后虽然能够终止，然而很难命中光源！
 - Next Event Estimation: 对光源采样
- 此前，我们对球面方向均匀采样.....然而，我们并不一定只能采用均匀采样策略！
 - 回忆：我们如何在Whitted-Style光线追踪中实现阴影？
 - shadow ray: 对光源采样



Next Event Estimation

- 如何对光源采样？
- 对光源采样，实际上是对面积采样.....
- 然而，我们现在的积分考虑入射方向微元 $d\omega_i$;
- 因此，我们需要考虑面积微元 dA ，也即建立 $d\omega_i$ 和 dA 的关系
- 回忆：Lecture 3 辐射度量学与BRDF（p9）

- 立体角 ω 具有如下微分形式：

$$d\omega = \frac{dA}{r^2}$$

面积微元
半径平方

需要是投影面积，因此下一页公式中会有一个余弦项， $d\omega = dA \cos\theta' / r^2$



Next Event Estimation

Sampling the Light

Then we can rewrite the rendering equation as

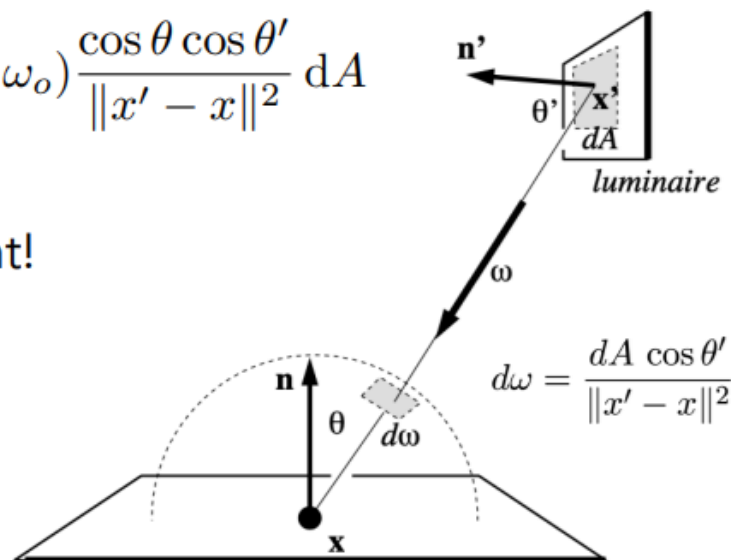
$$\begin{aligned} L_o(x, \omega_o) &= \int_{\Omega^+} L_i(x, \omega_i) f_r(x, \omega_i, \omega_o) \cos \theta \, d\omega_i \\ &= \int_A L_i(x, \omega_i) f_r(x, \omega_i, \omega_o) \frac{\cos \theta \cos \theta'}{\|x' - x\|^2} \, dA \end{aligned}$$

Now an integration on the light!

Monte Carlo integration:

"f(x)": everything inside

Pdf: $1 / A$



图片来源: [GAMES101 Lec16](#)



Next Event Estimation

➤NEE: 让路径中的每个中间节点都直接与光源上最终采样点相连, 得到不同长度路径

- 并让所有这些路径都参与计算

- 原做法, 得到1条采样路径:

- $x_k x_{k-1} \dots x_1 x_0$, 长度为k

- 新做法, 得到k-1条采样路径:

- $x_k x_{k-1} x'_0$, 长度为2

直接光照

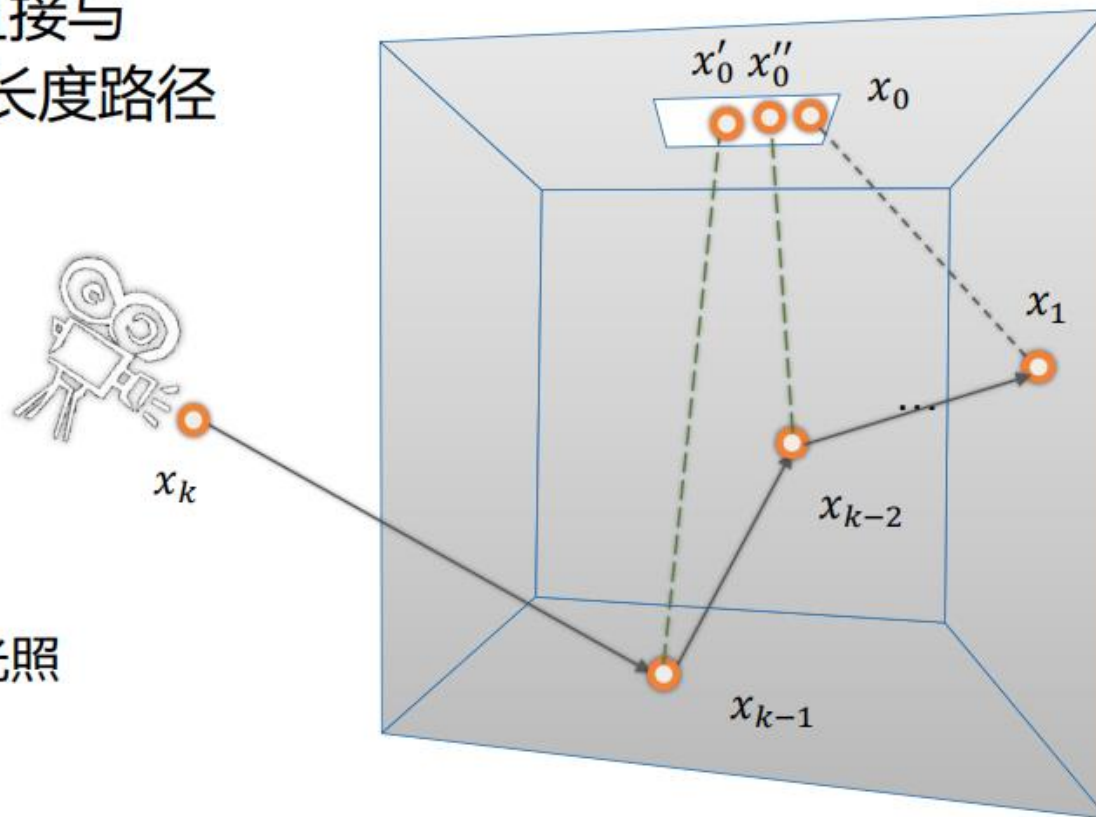
- $x_k x_{k-1} x_{k-2} x''_0$, 长度为3

...

- $x_k x_{k-1} \dots x_1 x_0$, 长度为k

- 注意, 一般做法是:

每条新采样路径上的光源采样点 x_0 每次都要重新采样





Next Event Estimation

```
shade(p, wo)

# Contribution from the light source.

Uniformly sample the light at x' (pdf_light = 1 / A)
L_dir = L_i * f_r * cos  $\theta$  * cos  $\theta'$  / |x' - p|^2 / pdf_light

# Contribution from other reflectors.

L_indir = 0.0

Test Russian Roulette with probability P_RR

Uniformly sample the hemisphere toward wi (pdf_hemi = 1 / 2pi)

Trace a ray r(p, wi)

If ray r hit a non-emitting object at q
    L_indir = shade(q, -wi) * f_r * cos  $\theta$  / pdf_hemi / P_RR

Return L_dir + L_indir
```

图片来源: [GAMES101 Lec16](#)



主要内容

- PA1: 光线投射
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分: 蒙特卡洛积分
 - 终止: RR (Russian Roulette)
 - 对光源采样: NEE (Next Event Estimation)
 - 复杂采样方法: MIS (多重重要性采样)
- 大作业说明
 - 得分项简述
 - 评分细则



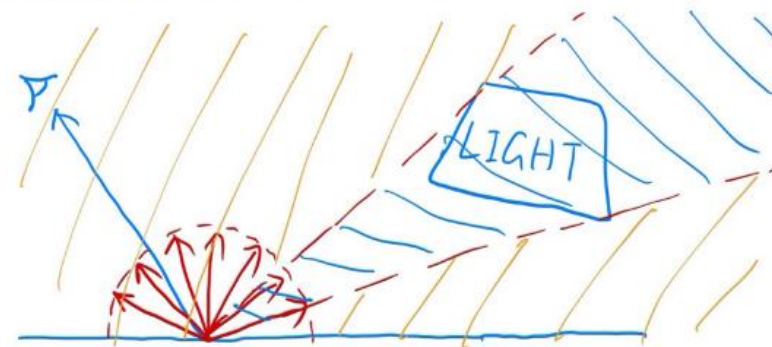
MIS（多重重要性采样）*

- 上述NEE实现中，考虑的是将渲染方程积分的积分域分为两部分分开考虑：

- 直接来自于光源的部分：对光源采样
- 不直接来自于光源的部分：均匀采样

1. light source (direct, no need to have RR)

2. other reflectors (indirect, RR)



- 然而，均匀采样时，也有可能采样到光源，此时需要丢弃该光线
- 是否可以更充分利用这些光线？

图片来源：[GAMES101 Lec16](#)



MIS（多重重要性采样）*

- 问题：直接来自于光源的部分中，使用了两种采样策略对积分进行估计
- 对于这部分，能否让两种采样策略共存？
- 多重重要性采样：

1. light source (direct, no need to have RR)

2. other reflectors (indirect, RR)



$$I_{mis}^{M,N} = \sum_{i=1}^M \frac{1}{N_i} \sum_{j=1}^{N_i} w_i(x_{ij}) \frac{f(x_{ij})}{p_i(x_{ij})}$$

$$\sum_{i=1}^M w_i(x) = 1$$

- M为采样策略数量；Ni为第i种采样策略的样本数
- 本质：用权重函数 $w_i(x)$ ，把积分拆分，不同部分交由不同策略估计



MIS（多重重要性采样）*

- 多重重要性采样：

$$I_{mis}^{M,N} = \sum_{i=1}^M \frac{1}{N_i} \sum_{j=1}^{N_i} w_i(x_{ij}) \frac{f(x_{ij})}{p_i(x_{ij})} \quad \sum_{i=1}^M w_i(x) = 1$$

- M为采样策略数量；Ni为第i种采样策略的样本数
- 本质：用权重函数 $w_i(x)$ ，把积分拆分，不同部分交由不同策略估计
- 权重函数的选取：可以使用balance heuristic

$$w_k(x) = N_k p_k(x) / \sum_{i=1}^M N_i p_i(x)$$



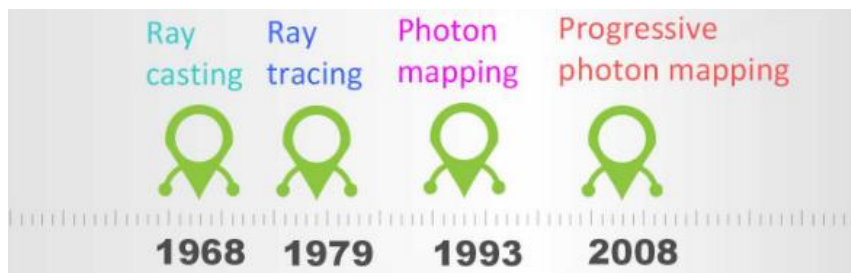
主要内容

- PA1: 光线投射
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分: 蒙特卡洛积分
 - 终止: RR (Russian Roulette)
 - 对光源采样: NEE (Next Event Estimation)
 - 复杂采样方法: MIS (多重重要性采样)
- 大作业说明
 - 得分项简述
 - 评分细则



光线跟踪算法

- 1968: 光线投射(Ray casting)
- Whitted-style光线跟踪(Whitted-style Ray Tracing)
- 路径追踪(Monte Carlo Ray Tracing / Path Tracing)
- 光子映射(Photon Mapping)——不作为大作业选项
- 双向路径跟踪(BDPT, Bidirectional Path Tracing)
- VCM(Vertex Connection and Merging)算法
- 路径引导(Path Guiding)





主要考核点

- 基本：NEE（Next Event Estimation）路径追踪引擎
 - 支持反射、折射、glossy材质，使用Russian Roulette终止
 - PBR（Physically-Based Rendering）：蒙特卡洛法解渲染方程
 - 支持面光源
- 加分：
 - 复杂光线追踪算法，复杂采样方案，参数曲面的渲染（Bezier曲面，B样条曲面），算法加速，额外效果等等



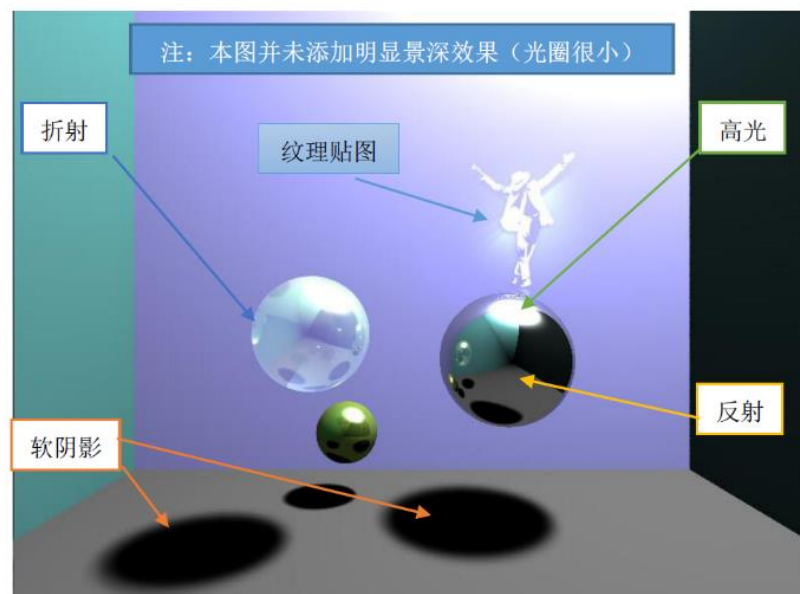
加分项总述

- 复杂算法选型（VCM, BDPT, ReSTIR DI/PT, **DDGI**变体...）
 - **光子映射类算法不得分**，包括PM、PPM、SPPM等
- 复杂网格求交加速（包围盒+树结构）
- 复杂采样策略（Path Guiding, Cos-weighted Sampling, BRDF Sampling, MIS等）
- 后处理：抗锯齿（4x MSAA、FXAA等）、超分辨率、**伽马校正**、**风格化渲染**等
 - **SSAA不得分**
- 景深（模拟相机焦距）、运动模糊（模拟曝光时间）
- 纹理贴图、法线贴图、位移贴图、Skybox（**环境光照贴图-对miss的光线作处理**）
- **各种效果对比**（参见大作业文档）
- 其他（蒙特卡洛体渲染、**Levels of Detail**等）



要求

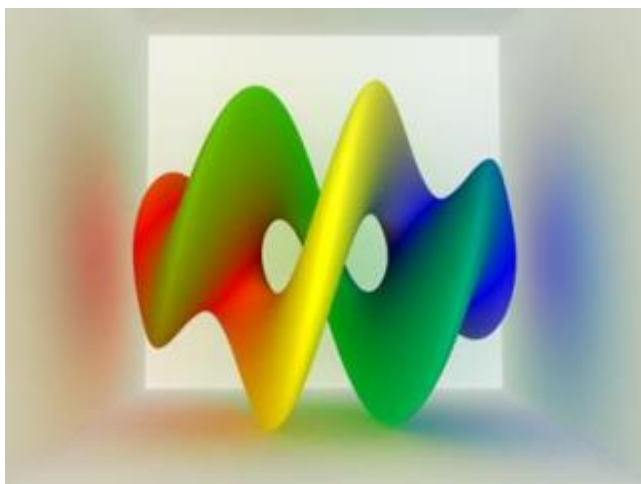
- 按时完成，可查阅开源资料和往届文档，不允许裸抄
- CPU上实现
- 至少实现Whitted-Style光线跟踪（反射+折射+阴影）
- 图片需要大于480P（640x480），建议使用无损png等格式进行存储。
 - 不建议在多张图上分别实现多个效果
- 严禁使用PhotoShop及其他画图工具进行后处理。
- 不要在最终结果上疯狂注释。
- 不要构造难以辨认的场景。
 - 例如使用纹理贴图贴一张光线跟踪结果。





1. 算法选型

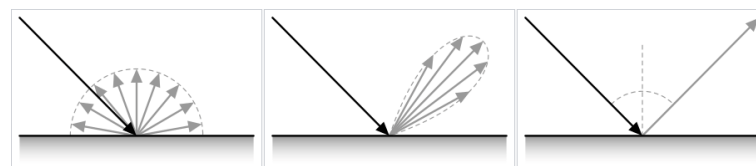
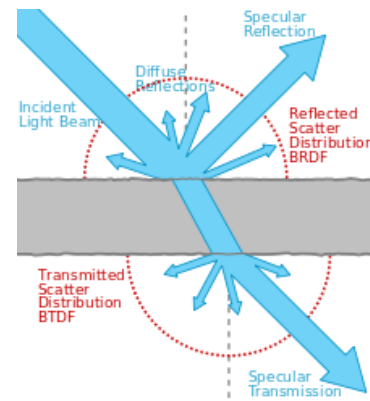
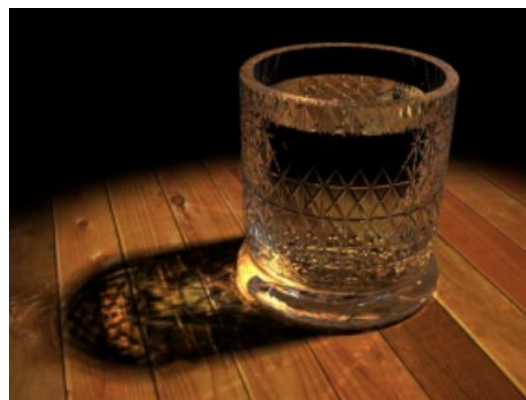
- 仅需实现一种渲染法。
 - 算法难度和运算量逐步提高。
 - 如果选择了较新的算法，应该体现出比老算法强在哪里。
 - 做Path Tracing: 漫反射，特殊材质下的Color Bleeding。
 - 做BDPT: Caustics – 由反射/透射引起的漫反射。
- 建议路径: Whitted-Style -> Path Tracing -> 1-2个加分项





算法选型

- Path Tracing
 - Color Bleeding
 - Physically Unbiased
- BiDirectional Path Tracing
 - Caustics
- DDGI（变体）：
 - 本质上是光栅化+光追混合策略，对漫反射表面直接查询irradiance缓存
 - 实现时不要求光栅化
 - 要求可视化Probe Texture





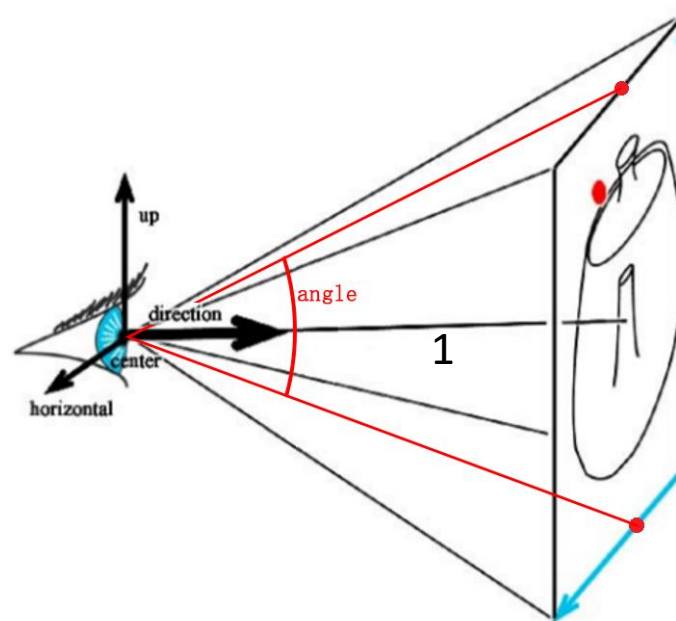
Whitted-Style Ray Tracing(单点光源)

```
RayTracer(Ray, Depth, weight, &Color) {  
    Color = 0;  
    if ( weight衰减到小于给定值 ) return; // 递归终止: 给定衰减阈值.  
    Color = Background;  
    计算Ray与场景中最近的物体的交点P; // 可以采用多种加速方法.  
    if ( 没有交点 ) return; // 递归终止: 返回背景色.  
    if ( P非阴影点 ) // P与光源间是否有物体阻挡.  
        Color = 局部光强; // 如: 书P149, 局部光强计算公式.  
    if ( Depth > 1 ) { // 递归终止: 一定次数.  
        if ( 当前面是镜面 ) {  
            计算反射光线ReflectedRay;  
            RayTracer(ReflectedRay, Depth-1, weight*w_r, &RefColor);  
            Color += w_r * RefColor;  
        }  
        if ( 当前面是透射面 ) {  
            计算透射光线TransmittedRay;  
            RayTracer(TransmittedRay, Depth-1, weight*w_t, &TransColor);  
            Color += w_t * TransColor;  
        }  
    }  
}
```




Path Tracing Algorithm

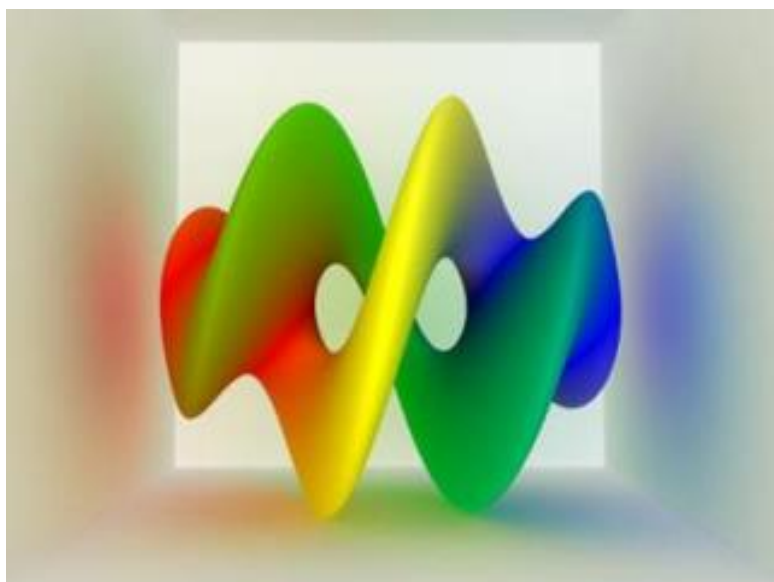
```
1: for each pixel (i,j) do
2:   Vec3  $C = 0$ 
3:   for (k=0; k < samplesPerPixel; k++) do
4:     Create random ray in pixel:
5:       Choose random point on lens  $P_{lens}$ 
6:       Choose random point on image plane  $P_{image}$ 
7:        $D = \text{normalize}(P_{image} - P_{lens})$ 
8:       Ray ray = Ray( $P_{lens}$ ,  $D$ )
9:     castRay(ray, isect)
10:    if the ray hits something then
11:       $C += \text{radiance}(\text{ray}, \text{isect}, 0)$ 
12:    else
13:       $C += \text{backgroundColor}(D)$ 
14:    end if
15:  end for
16:  image(i,j) =  $C / \text{samplesPerPixel}$ 
17: end for
```



Path Tracing



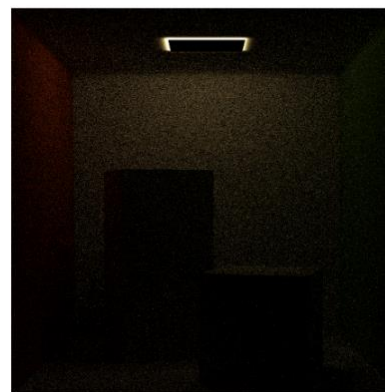
双向路径跟踪会从相机和光源处出发，形成两条路径，之后会枚举两条路径的长度和，并对所有可行的路径进行合并并使用mis策略。以下两个场景展示了双向路径跟踪在光源特别难采样时的效果（光源颠倒了）。



(f) mis, 48spp, 10.6s



(g) bpt, 16spp, 9.2s



(h) mis, 1024spp



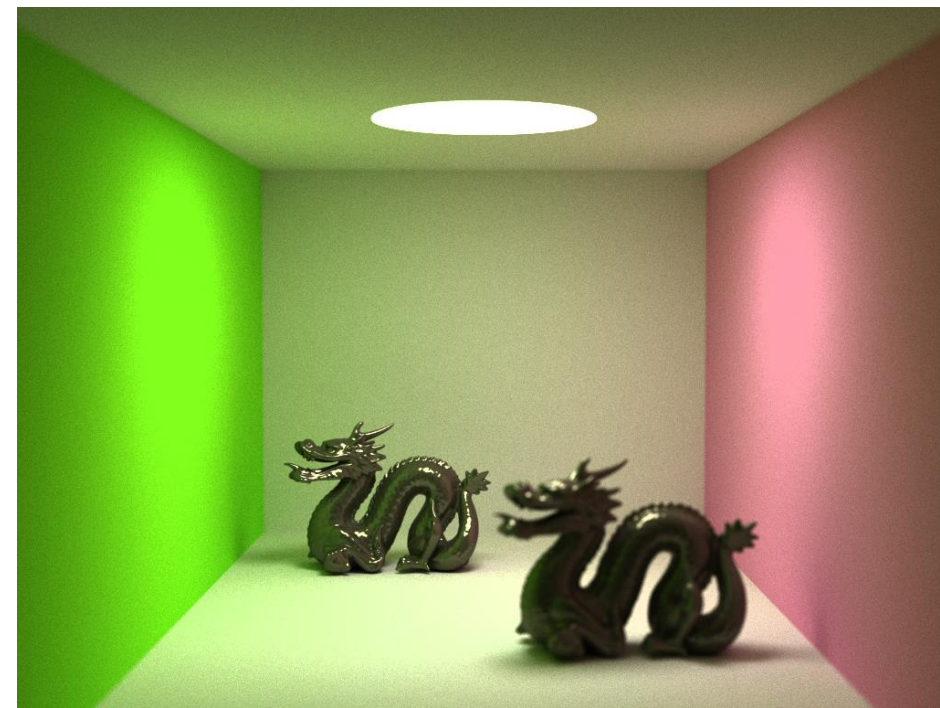
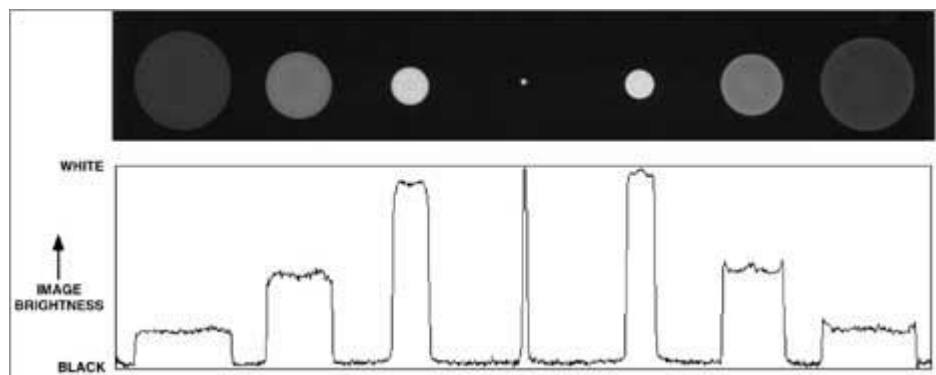
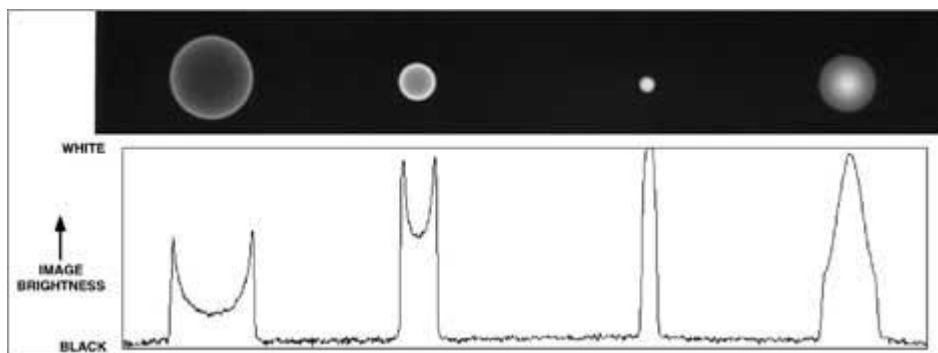
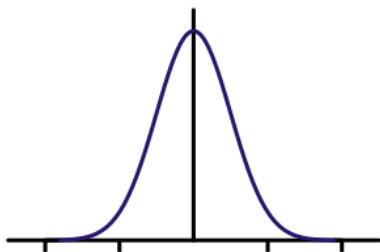
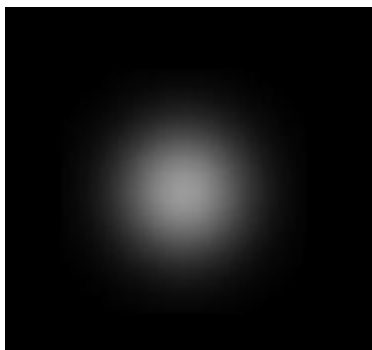
2. 时间代价

- 最终评分主要看效果，做的加速(包围盒、包围球、Octree、Kd-tree)请在报告中注明。
- 参数曲面（尤其是高次）和复杂网格求交的时间代价比一般曲面要高很多，完成作业时注意留足渲染时间。
- 多项附加功能会导致渲染时间乘法式叠加，夜里挂程序请保护计算机的安全。



3. 分布式光线追踪（景深、运动模糊etc.）

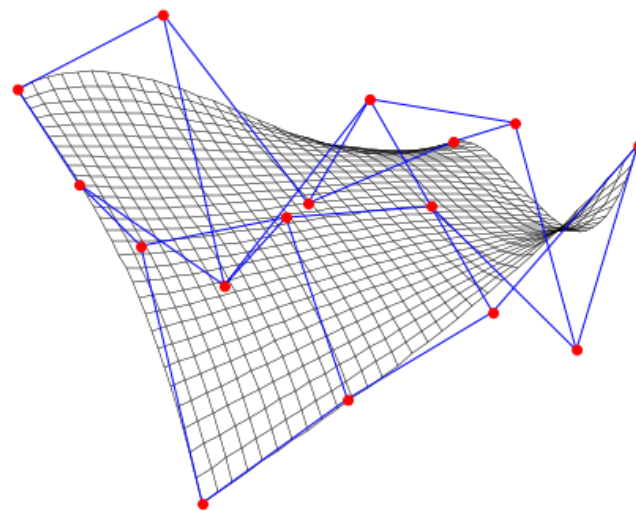
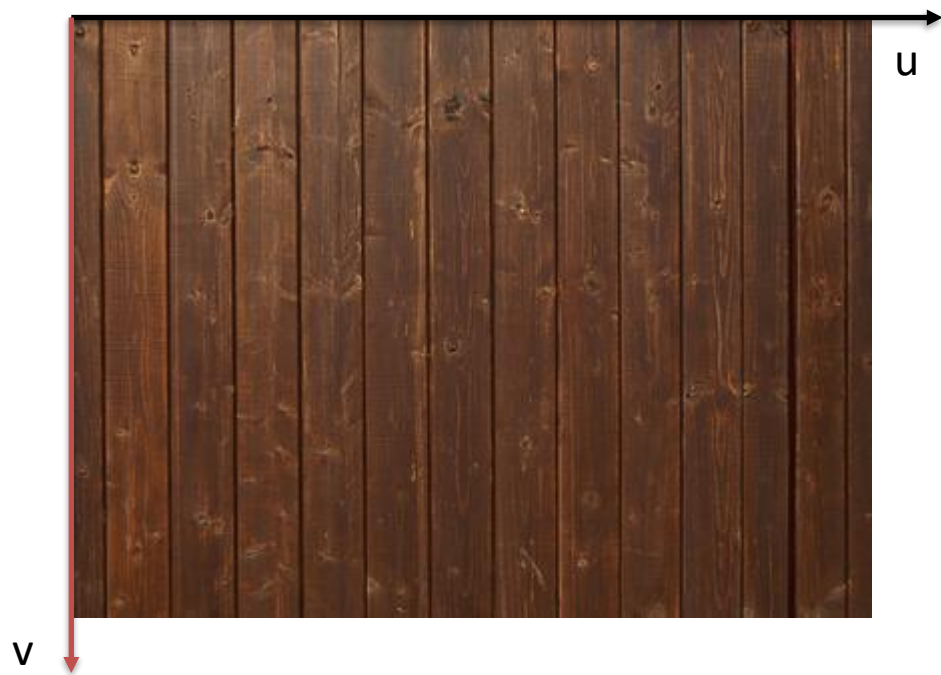
- 景深三要素：光圈、焦距、物距





4. 纹理映射

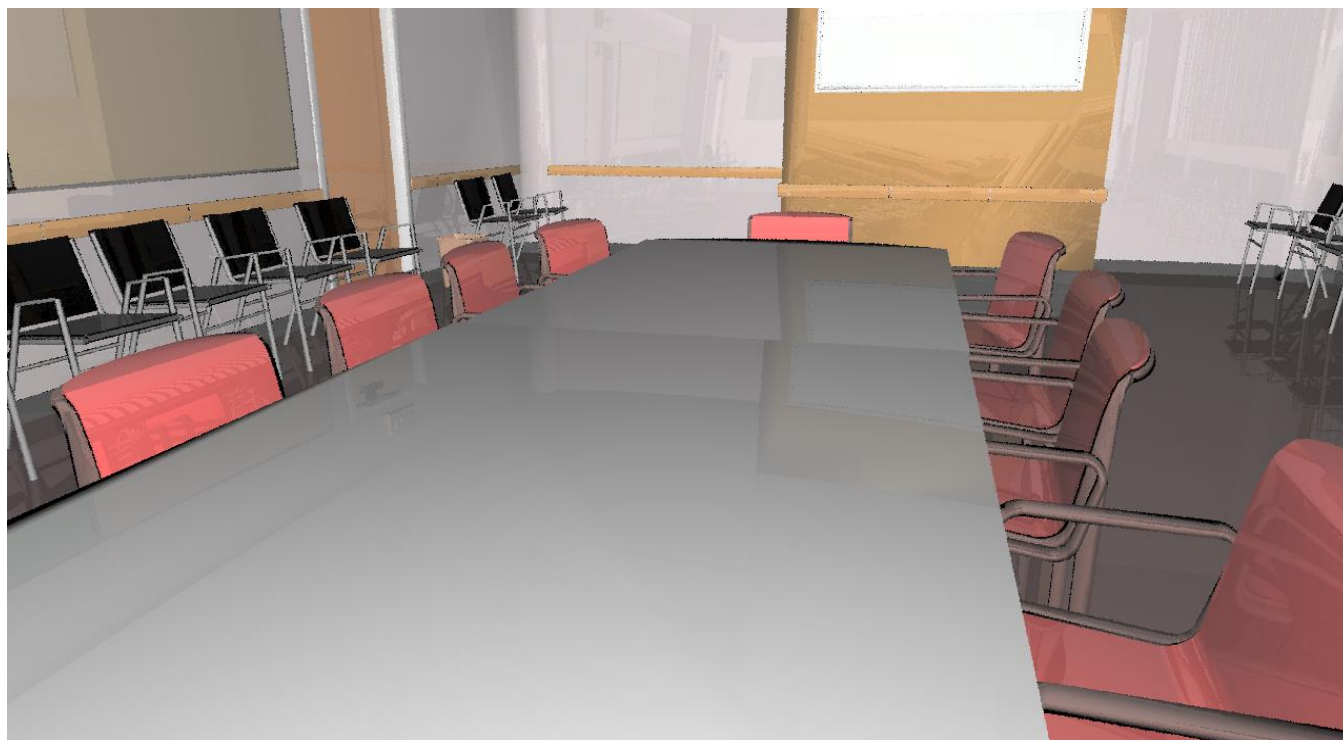
- 与参数曲面求交时，UV纹理坐标和求出来的参数值完美对应。
- 一般的面片，也可以将求交结果转化为参数坐标后读出需要取的纹理颜色值。





5. 复杂网格/场景

- 建议做1~2个参数曲面或复杂网格模型即可。





5. 复杂网格/场景

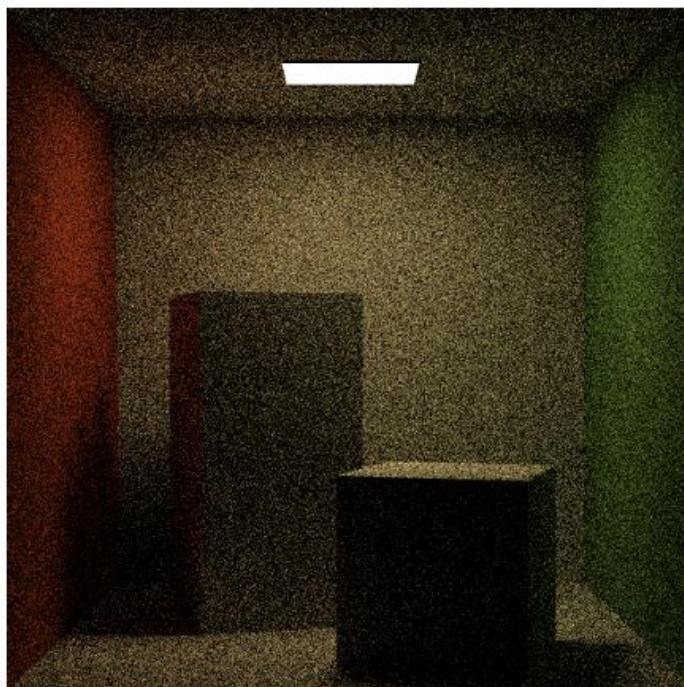
- 建议做1~2个参数曲面或复杂网格模型即可。



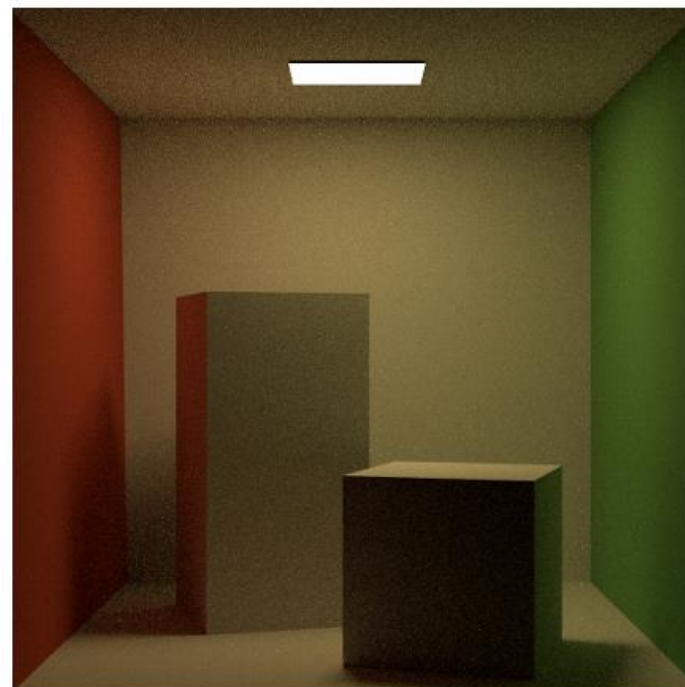


6. 效果对比

- 例：使用NEE与不使用NEE



(a) naive, 128spp, 28s

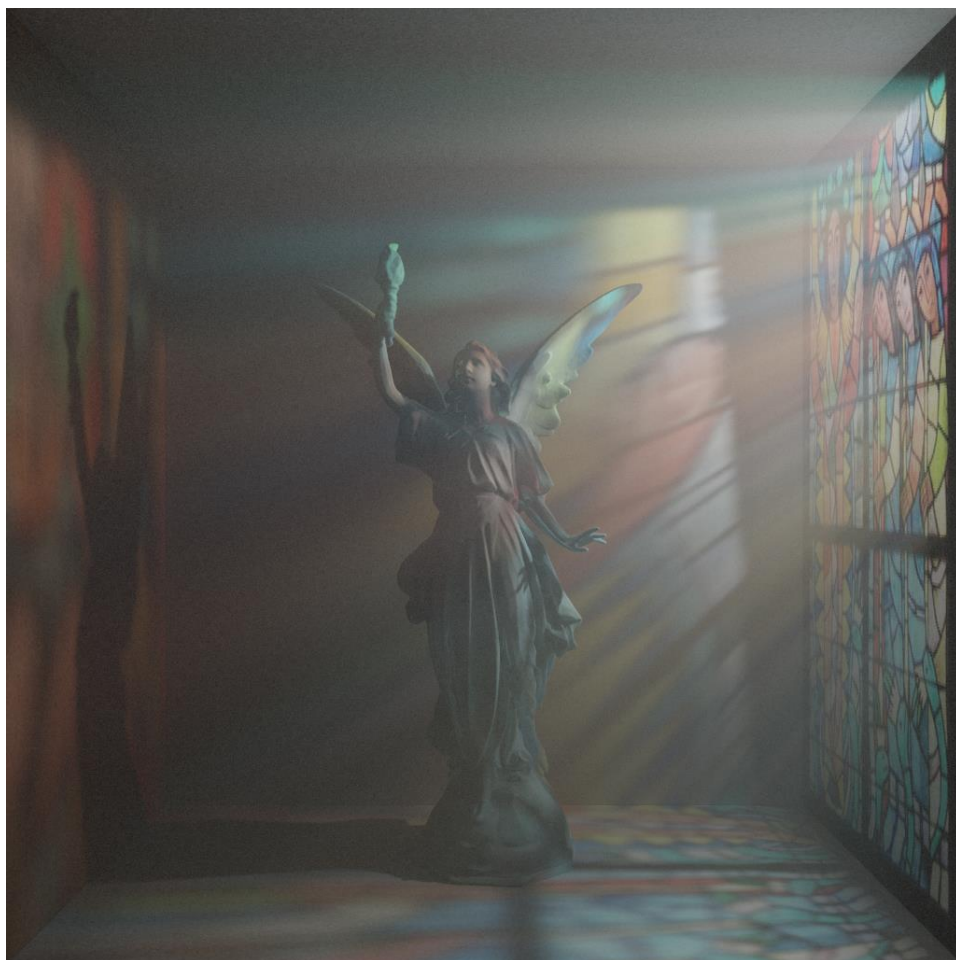


(b) nee, 16spp, 4s



X. 蒙特卡洛体渲染

- 体积光：烟/雾渲染





主要内容

- PA1: 光线投射
 - 算法概述
 - 代码讲解
- 路径追踪
 - PBR中如何计算渲染方程积分: 蒙特卡洛积分
 - 终止: RR (Russian Roulette)
 - 对光源采样: NEE (Next Event Estimation)
 - 复杂采样方法: MIS (多重重要性采样)
- 大作业说明
 - 得分项简述
 - 评分细则



最终提交和验收

- Result = 一张/多张渲染好的图片
- Code
 - 会进行查重，自己写的代码需要标注清晰（按文件/函数/代码块标注）；参考需要在报告中标明来源
 - 基于光子映射的算法不得分，非基于PA1进行开发的代码加分项不得分
- Report
 - 开头应明确列出自己的所有得分点，避免遗漏。
 - 再分别列出每个得分点对应的代码段，还可以辅以说明，方便查验。
 - 需要对实现的功能作分析对比（见大作业文档）。
 - 报告最后再附上实验结果图片。
- 期末时进行当面验收，原则上不允许远程验收和推迟。
 - 验收以交流为主，验收时用的图像不作为最终评分标准
 - 之后可以补交最终结果。



评分细则（占总评45分）

- 基本实现
 - 实现Whitted-Style光线追踪（反射、折射、阴影，有明显bug扣分）
 - 实现路径追踪（体现间接光照）
 - 漫反射间接光照使用均匀采样计算，支持面光源，路径终止使用Russian Routelette
 - PBR: Phong着色->基于物理的着色，支持glossy BRDF
 - NEE（Next Event Estimation）
 - 主观分: 设计和构图
- 加分项
 - 更复杂的光线追踪方案：双向光线跟踪算法(BDPT)、ReSTIR、VCM算法、DDGI变种等
 - 更复杂的采样方法：Path Guiding，BRDF采样，MIS等
 - 光线求交：
 - 复杂网格模型及其加速结构
 - 参数曲面解析法求交
 - 其他效果：景深/抗锯齿/贴图/体积光等

习题课 实现效果对比与分析



- 评分
 - 实现的功能：多写不扣分
 - 功能的完成度和呈现质量：影响每项功能的具体得分
 - 整体渲染效果
 - 实验报告：按要求描述清楚即不会扣分



参考资料

- smallpt (cos-weighted采样) : [smallpt: Global Illumination in 99 lines of C++ \(kevinbeason.com\)](#)
- 超分辨率: [FuseSR \(Siggraph Asia 2023\)](#)
- [NEE with MIS的一个参考实现](#)
- MIS: [veach-chapter9.pdf \(stanford.edu\)](#)
- BDPT: [veach-chapter10.pdf \(stanford.edu\)](#)
- 蒙特卡洛体渲染: [Monte Carlo methods for physically based volume rendering](#)



参考资料

- 风格化渲染: [Stylized Rendering as a Function of Expectation | ACM Transactions on Graphics](#)
- Path Guiding: [Path guiding in production | ACM SIGGRAPH 2019 Courses](#)
- [【物理渲染】基础知识 与 Cook-Torrance BRDF 模型](#)
- 景深、运动模糊: [Distributed ray tracing](#)
- DDGI: [Dynamic Diffuse Global Illumination with Ray-Traced Irradiance Fields](#)



Thank You !
Any Questions?